A BEGINNER'S SESSION • CORE JAVA

Java Fundamentals

Introduction • JDK • JVM • Program Structure • Hello World

C vs Java • Printing Messages • Program Execution

PRESENTED BY

Varun Masih

```
public class JavaFundamentals {
    public static void main(String[] a) {
        System.out.println("Hello!");
```

Compiled with JDK 21 • every snippet in this deck was run

Java is a high-level, object-oriented, platform-independent language (James Gosling, Sun Microsystems, 1995).

1. Java — Introduction

What it is, who made it, why it is still everywhere

Java is a high-level, object-oriented, platform-independent language: you compile once to bytecode and run it anywhere a JVM exists.

- **Created by** James Gosling's team at Sun Microsystems; announced 23 May 1995 (named "Oak" first).
- **One promise — WORA:** "Write Once, Run Anywhere" — the same bytecode file runs on Windows, Linux, macOS and Android.
- **Owned by Oracle** since it acquired Sun in 2010; open-source under GPL via the OpenJDK project.
- **Still the workhorse:** Android apps, banking systems, government portals, Minecraft and most large web back-ends are Java.
- **Latest LTS:** Java 25 (16 September 2025); LTS line is 8, 11, 17, 21, 25 — enterprises upgrade slowly.
- **Core traits:** strongly typed, object-oriented, automatic garbage collection, huge library ecosystem (Maven, Spring).

Smallest real program — a class, a main method, two prints

```
// Java: not a script - a real program
public class Intro {
    public static void main(String[] args) {
        String lang = "Java";
        int year = 1995;
        System.out.println(lang + " was born in " + year);
        System.out.println("Same file runs on any JVM!");
    }
}

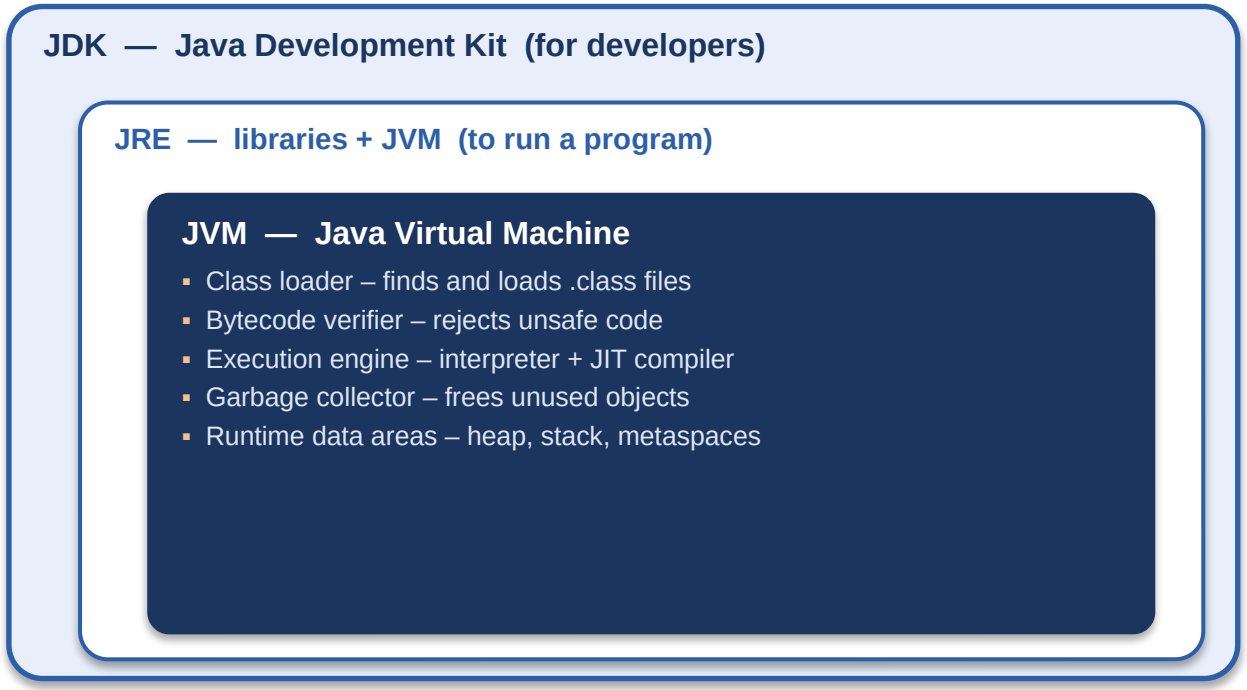
OUTPUT (ran with java 21):
Java was born in 1995
Same file runs on any JVM!
```

Real-world use case: A bank writes one payment service in Java, builds a single .jar, and deploys that same file on Windows servers, Linux cloud VMs and even a mainframe — no rewriting per platform.

2. JDK — Java Development Kit

The toolbox you install to write, compile and package Java

The JDK is the developer's toolkit: it contains the compiler javac plus the JRE (which contains the JVM), so one install gives you both building and running.



The four JDK tools you use first

```
javac Hello.java        # source -> bytecode
java  Hello             # start the JVM
javadoc -d docs Hello.java  # API docs
jar --create --file app.jar Hello.class
```

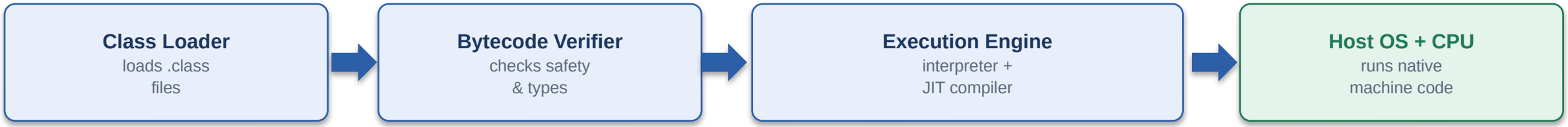
- **javac —** the compiler: turns .java text into .class bytecode.
- **java —** the launcher: starts a JVM and runs a class.
- **javadoc / jar —** build documentation and package classes into one .jar.
- **Since JDK 11 there is no separate JRE download —** the JDK includes it.

Real-world use case: A student installs one JDK (Temurin 21) and can compile class assignments, run them, and package the project into a .jar for submission — one download, everything needed.

3. JVM — Java Virtual Machine

The engine that turns bytecode into running machine code

The JVM is an abstract machine that loads .class bytecode, verifies it, executes it with an interpreter + JIT compiler and manages memory through garbage collection.



Runtime data areas (created by the JVM)



Bytecode is not machine code — it is JVM instructions

```
$ javac HelloWorld.java
$ javap -c HelloWorld     # read the bytecode
3: ldc           #13  // String "Hello, World!"
5: invokevirtual #15  // PrintStream.println
8: return
```

- **JIT hot-spot compilation —** frequently used methods are compiled to native code for near-C speed.
- **Garbage collection —** you never call free(); the JVM reclaims unreachable objects.
- **Platform translation —** one .class file, a different JVM per OS and CPU.

Real-world use case: The same compiled .class runs unchanged on an x86 cloud server, an Apple-silicon laptop and an Android phone — each device has its own JVM doing the translation.

4. Program Structure

The fixed skeleton every Java file follows

A Java program always follows the same order: optional package → imports → class declaration → fields and methods → the main method where the JVM starts.

Structure.java — saved as com/acme/demo/Structure.java

```
package com.acme.demo;        // 1  package
import java.util.Scanner;     // 2  imports
public class Structure {      // 3  class
    static final String APP = "Java Fundamentals";
    // 4  fields / constants
    public static void main(String[] args) {  // 5
        System.out.println(APP);  // 6  statements
    }
}

OUTPUT: Java Fundamentals
```

- **1. package —** the folder namespace (optional but standard in real projects).
- **2. import —** borrows classes from the JDK or your libraries.
- **3. class —** everything lives inside a class; the public class name must match the file name.
- **4. fields —** constants and variables that belong to the class.
- **5. main —** the JVM entry point: public static void main(String[] args).
- **6. statements —** end with ; and sit inside braces { }, one block per level.

Run it with the package name: java -cp out com.acme.demo.Structure

Real-world use case: Because every project follows this order, an IDE or a new teammate can open any file — say OrderService.java — and instantly find the package, the imports and the entry point.

5. Hello World

The smallest complete Java program — and the keywords inside it

Hello World is one class with one main method and one print statement — running it proves your JDK, classpath and JVM are all correctly installed.

HelloWorld.java

```
// file: HelloWorld.java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Write, compile, run — verified on JDK 21

```
$ javac HelloWorld.java
$ java HelloWorld   ->   Hello, World!
```

Every keyword explained

Keyword	What it does in one line
`public`	the class and method are visible to the JVM from anywhere
`static`	runs without creating an object of the class
`void`	the method hands nothing back to the JVM
`main`	the fixed name the JVM looks for to start the program
`String[] args`	holds command-line arguments typed after the class name

Real-world use case: CI pipelines run a Hello World as a smoke test: if it compiles and prints, the build machine's JDK, CLASSPATH and runtime are proven healthy before the real 200-file build starts.

6. C vs Java

Same job, two very different routes from source to output

C compiles straight into machine code for one platform and you manage memory yourself, while Java compiles to portable bytecode that the JVM runs and whose memory the garbage collector manages.

Aspect	C	Java
Compiled to	native machine code	platform-independent bytecode
Runs on	the OS directly	a JVM (inside the JRE)
Portability	recompile for each CPU/OS	one .class file everywhere
Memory	malloc() / free() by hand	garbage collector
Pointers	full pointer arithmetic	references only — no pointers
Style	procedural	object-oriented
Entry point	int main(void)	public static void main(String[])
Output file	hello / a.out (binary)	Hello.class (bytecode)

In C — you end the line, you return the status

```c
/* hello.c - compile:  gcc hello.c -o hello */
#include <stdio.h>
int main(void) {
    printf("Hello, World!\n");
    return 0;
}
```

In Java — the print goes through the JVM

```java
// Hello.java - compile: javac Hello.java
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Both printed the same line: **"Hello, World!"** — one via the CPU, one via the JVM.

Real-world use case: C is chosen for firmware, kernels and device drivers where raw speed and hardware access matter; Java is chosen for large business systems that must run on many servers unchanged.

7. Printing Messages

print, println and printf — and the newline escape

System.out.print keeps the cursor on the same line, println adds a line break after the text, and printf formats numbers and aligned text — all three write to standard output.

PrintDemo.java

```
public class PrintDemo {
    public static void main(String[] args) {
        System.out.print("Java ");   // no newline
        System.out.println("Fundamentals");
        System.out.println("line two");
        System.out.printf("Score: %.2f%n", 91.5);
        System.out.printf("%-8s|%5d%n", "Total", 42);
    }
}
```

- **print(...) —** writes the text and stays on the same line (the next print continues it).
- **println(...) —** writes the text and moves to the next line; println() alone makes a blank line.
- **printf("...%n", x) —** formats output: %.2f two decimals, %d integer, %s text, %-8s left-aligned.
- **%n vs \n —** %n is the platform-independent newline; \n is the classic escape inside a string.
- **Escapes you need:** \n newline, \t tab, \\ backslash, \" a quote inside a string.

Actual output — pasted from the terminal

```
Java Fundamentals
line two
Score: 91.50
Total   |   42
```

Mental model: System.out is the screen, the print methods are the pen — print = write on, println = write + Enter, printf = write on a stencil.

Real-world use case: Billing software prints invoices with printf so that every amount lines up in the same column (₹ 1,250.00) and every log line carries the same timestamp format.

8. How a Java Program Executes

From the file you type to the output you see

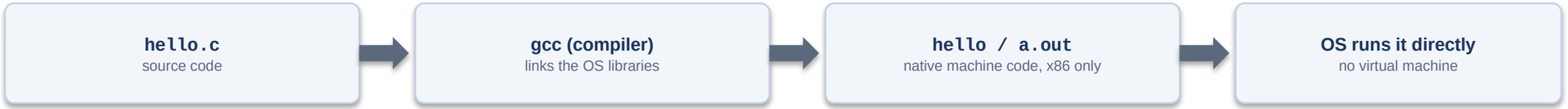
Execution happens in two stages: javac compiles source code into bytecode, then the JVM loads, verifies, interprets and JIT-compiles that bytecode so the host OS can run it.

Stage 1 + 2 — compile, then run (Java)



Write once → compile once → the same .class runs on every OS with a JVM

The same program in C — one route, one platform




```
$ javac HelloWorld.java   # .java  -> .class
$ java  HelloWorld        # JVM loads & runs it
Hello, World!
```

- **Compile-time errors** appear at javac (typos, wrong types) — the program never starts.
- **Runtime errors** appear while the JVM executes (e.g. ArithmeticException on / 0).

Real-world use case: A team compiles once on the build server and ships the same .jar to Windows, Linux and macOS production machines — only the JVM differs, the bytecode does not.

Recap — Java Fundamentals on One Page

Every topic in a single line

Java
High-level, object-oriented, platform-independent language (Gosling, 1995).

JDK
Developer toolkit: javac + javadoc + jar + the JRE, which contains the JVM.

JVM
Loads, verifies and runs bytecode with interpreter, JIT and garbage collection.

Structure
package → imports → class → fields → methods → main(String[] args).

Hello World
One class, one main method, one println — proves the toolchain works.

C vs Java
C: native code + manual memory; Java: portable bytecode + automatic GC.

Printing
print stays on the line, println breaks it, printf formats the values.

Execution
javac makes bytecode, the JVM runs it — compile once, run anywhere.

Next steps: write your own Hello World, then change the text with a String variable and a command-line argument.